PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification

for

ChatSphere – Real-Time Direct & Group Messaging Platform

Version 1.2 approved

Prepared by Team 16 – Satvik L, Prithvi Aithal, Sourish Ganesh
PES University, Bangalore – Dept. of CSE, Section G

Team Members and SRNs

Sl.	Student Name	SRN	USN
1	Satvik L	PES1UG24CS423	PES1202400676
2	Prithvi Aithal	PES1UG24CS909	PES1202400699
3	Sourish Ganesh	PES1UG25CS847	PES1202503600

September 2026

Revision History

Name	Date	Reason For Changes	Version
Team 16	Sep 2026	Initial SRS baseline for Requirements submission	1.0
Team 16	Sep 2026	Added security objectives and expanded security requirements	1.1
Team 16	Nov 2026	As-built update: Appendix D records the changes made during implementation (deviations D-01 to D-14) and the verification results	1.2

Table of Contents

Revision History 2
Table of Contents 3
1. Introduction 5
1.1 Purpose 5
1.2 Intended Audience and Reading Suggestions 5
1.3 Product Scope 5
1.4 References 5
2. Overall Description 5
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 6
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 6
2.6 Assumptions and Dependencies 6
3. External Interface Requirements 6
3.1 User Interfaces 6
3.2 Software Interfaces 7
3.3 Communications Interfaces 7
4. Analysis Models 7
5. System Features 8
5.1 Authentication & Presence Management 8
5.2 Direct Messaging 9
5.3 Group / Room Chat 10
5.4 Message History & Status Tracking 10
5.5 Room Administration 10
6. Other Nonfunctional Requirements 11
6.1 Performance Requirements 11
6.2 Safety Requirements 11
6.3 Security Objectives and Requirements 11
6.4 Software Quality Attributes 12
6.5 Business Rules 12
7. Other Requirements 12
8. Validation Specification – Initial Planning 12
9. Project Plan 13
9.1 Work Breakdown Structure (WBS) 13
9.2 Schedule (Sprint / Gantt Summary) 13
9.3 Tool Stack 14
Appendix A: Glossary 14
Appendix B: Field Layouts 14
Appendix C: Requirement Traceability Matrix 15

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of ChatSphere, a real-time direct and group messaging web application built as the Level 3 Software Engineering mini-project (UE24CS341A) by Team 16. It covers the complete set of functional, non-functional and security requirements for the initial release and is the baseline against which the system's design, implementation and testing will be validated.

1.2 Intended Audience and Reading Suggestions

This SRS is intended for: the development team (Satvik L, Prithvi Aithal, Sourish Ganesh), who use it as the contract for design and implementation; the course faculty/evaluators, who use it to assess requirement quality and traceability; and the team members acting as testers, who derive test cases directly from Sections 5 and 6. Readers unfamiliar with the domain should first read Section 2 (Overall Description) before the detailed feature requirements in Section 5.

1.3 Product Scope

ChatSphere provides registered users with low-latency, one-to-one and group (room-based) text messaging over a persistent WebSocket connection, including real-time presence, typing indicators, delivery/read status and persisted, searchable message history. Its purpose is both to give a small community (e.g. a project team or campus group) a self-hostable communication tool, and to demonstrate the team's understanding of socket programming, event-driven architecture and the full software development life cycle.

1.4 References

- PES University, Dept. of CSE – “Jackfruit – Software Engineering Project Guidelines 2026” (UE24CS341A).
- PES University – “Software Engineering – Level 3 Student Submission Guide & Complete Exemplar” (Coffee Kiosk).
- Karl Wiegers – IEEE-style Software Requirements Specification template (structure followed by this document).
- Socket.IO official documentation – https://socket.io/docs/
- Express.js official documentation – https://expressjs.com/

2. Overall Description

2.1 Product Perspective

ChatSphere is a new, self-contained, three-tier web application; it is not a follow-on to an existing system. It consists of a React single-page-application client, a Node.js/Express + Socket.IO server that exposes both REST endpoints (for auth and history retrieval) and a WebSocket gateway (for real-time events), and a persistence tier composed of MongoDB (durable data – users, rooms, messages) and Redis (ephemeral presence/session state). The high-level flow is: Client ↔ REST API / WebSocket Gateway ↔ Service Layer ↔ MongoDB / Redis.

2.2 Product Functions

- User registration and secure (JWT-based) login.
- Real-time one-to-one direct messaging.
- Room creation, joining and leaving for group chat.
- Real-time group messaging within a room.
- Online/offline presence broadcasting.
- Real-time typing indicators.
- Persistent, paginated message history retrieval.

- Room administration – promoting or removing a member.
- Message delivery/read status tracking.

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Registered User	Register/login; send & receive direct and group messages; create/join/leave rooms; view own conversation history; view presence	Non-technical, expects near-instant delivery; frequent short sessions from browser
Room Admin	All Registered User privileges, plus: promote another member to admin, remove a member from a room they administer	Same technical profile as Registered User; privilege is scoped only to rooms they created or were promoted in

2.4 Operating Environment

- Client: any evergreen desktop browser with WebSocket support (Chrome, Firefox, Edge).
- Server: Node.js 20 LTS runtime on Linux, packaged as a Docker container.
- Database: MongoDB 6+; Redis 7+ for presence/session pub-sub.
- Network: HTTPS for REST calls, WSS (WebSocket Secure) for the real-time channel.

2.5 Design and Implementation Constraints

- Must use WebSocket-based real-time communication (Socket.IO) per the course assignment – polling-only designs are not acceptable.
- Agile methodology is mandatory; work must be tracked as user stories/sprints on GitHub Projects (or Jira).
- Git/GitHub is the system of record for version control; individual Git commits are checked as evidence of contribution.
- A CI/CD pipeline, code reviews and security validation are mandatory course deliverables for later phases.
- Team of exactly 3 members; each member must own one complete functional feature end-to-end (Section 9 of the Synopsis).

2.6 Assumptions and Dependencies

- Users have a reasonably stable internet connection; brief disconnects (under ~60s) are expected to resume without message loss, but ChatSphere does not guarantee delivery over extended offline periods beyond queued-message retry.
- The project depends on third-party open-source libraries (Express, Socket.IO, Mongoose, JWT, bcrypt) remaining available and API-stable for the duration of the project.
- This phase targets a single-server deployment; multi-instance/horizontal scaling of the Socket.IO layer is out of scope (see Synopsis Section 7).

3. External Interface Requirements

3.1 User Interfaces

- Registration / Login screen – username, email, password fields, client-side validation and error messaging.
- Chat Dashboard – left panel listing direct conversations and joined rooms with unread counts and presence dots; right panel showing the active conversation.
- Message Composer – text input with a live typing indicator broadcast to the other participant(s), and per-message delivery/read ticks.
- Room Panel – member list with roles, and (for a Room Admin) controls to promote or remove a member.
- Search – a search box to find existing users or rooms to start a new conversation.

3.2 Software Interfaces

- REST API (JSON over HTTPS) – used for registration, login, and paginated history retrieval; implemented with Express.js.
- WebSocket API (Socket.IO over WSS) – used for all real-time events: message send/receive, typing, presence, room join/leave, room-admin actions.
- MongoDB via the Mongoose ODM – durable storage for User, Room, RoomMembership and Message documents.
- Redis – pub/sub channel for presence propagation and Socket.IO adapter for cross-instance event broadcast (forward-compatibility for future scaling).

3.3 Communications Interfaces

- All REST traffic is served over HTTPS; all real-time traffic is served over WSS (WebSocket Secure).
- Messages exchanged between client and server are JSON-encoded.
- Authentication uses JSON Web Tokens (JWT) sent as a Bearer token on REST calls and as an auth payload during the Socket.IO handshake.

4. Analysis Models

The use case diagram below identifies the two actors (Registered User and the more privileged Room Admin) and the major use cases they participate in.

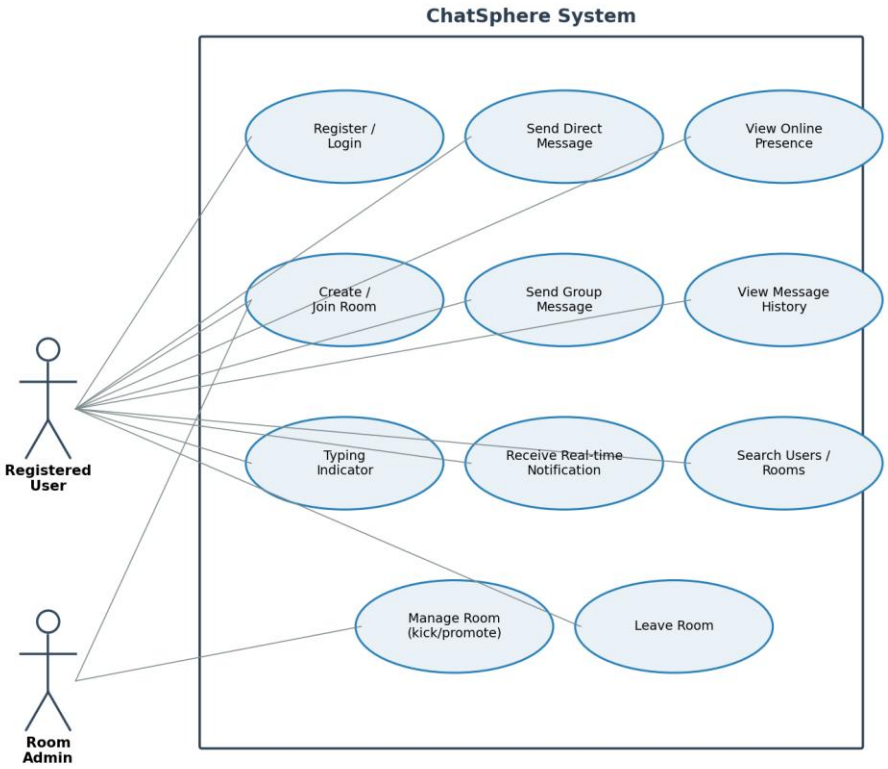


Figure 1 – Use Case Diagram for ChatSphere

The entity-relationship model below shows the core persisted entities – User, Room, RoomMembership (the join entity carrying each member's role) and Message – plus the ephemeral Session entity used to map a live socket connection back to a user.

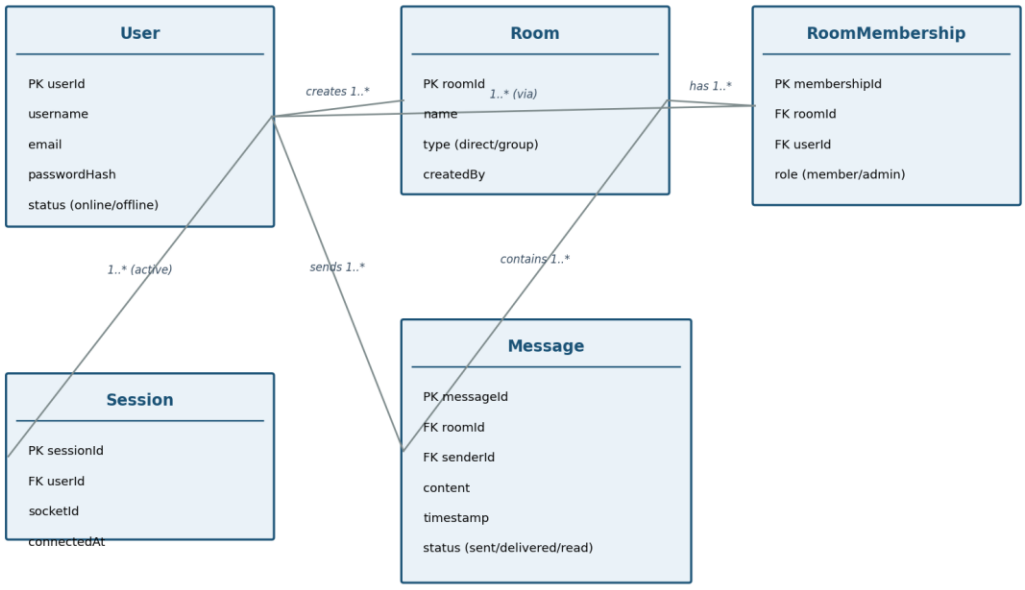


Figure 2 – Entity-Relationship Diagram for ChatSphere's data model

Use Cases

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register / Login	Registered User	Application reachable	Authenticated session (JWT) issued
UC-02	Send Direct Message	Registered User	Logged in	Message delivered (or queued if recipient offline)
UC-03	Create / Join Room	Registered User	Logged in	Room membership created
UC-04	Send Group Message	Registered User	Room member	Message broadcast to all online room members
UC-05	View Message History	Registered User	Conversation/room exists	Paginated history returned
UC-06	Manage Room	Room Admin	Admin authenticated	Member role updated / member removed
UC-07	View Presence	Registered User	Logged in	Online/offline status of contacts shown

5. System Features

Requirements are organized by system feature. Each feature lists a description/priority, its stimulus/response behaviour, and its detailed, uniquely-tagged functional requirements. Each requirement is concise, unambiguous, verifiable and necessary, per the course's requirement-writing rule.

5.1 Authentication & Presence Management

Description and Priority

Lets a person register a ChatSphere account, log in to obtain a session, and lets the system reflect that user's connection state to others in real time. Priority: High – every other feature depends on it. Owned by Sourish Ganesh.

Stimulus/Response Sequences

- User submits the registration form → system creates the account or returns a validation error.
- User submits valid credentials → system returns a JWT and the client opens an authenticated socket connection.
- Client's socket connects/disconnects → system broadcasts the user's presence change to relevant contacts/rooms.

Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a new user to register using a unique username and a unique email address.	High	Functional test
FR-002	The system shall hash user passwords (bcrypt) before storage; plaintext passwords shall never be persisted.	High	Code/config review
FR-003	The system shall allow a registered user to log in with valid credentials and receive a signed JWT session token.	High	Functional test
FR-004	The system shall authenticate the WebSocket handshake using the JWT issued at login and reject the connection if the token is missing, expired or invalid.	High	Security test
FR-005	The system shall broadcast a user's online status to their direct contacts and room members immediately upon socket connection.	High	Integration test
FR-006	The system shall broadcast a user's offline status to their direct contacts and room members within 5 seconds of socket disconnection.	Medium	Integration test

5.2 Direct Messaging

Description and Priority

Lets any two registered users exchange private, real-time text messages with typing and delivery-status feedback. Priority: High. Owned by Prithvi Aithal.

Stimulus/Response Sequences

- User selects a contact and sends a message → system delivers it instantly if the recipient is online, otherwise queues it.
- User starts typing in an open conversation → system notifies the other participant in real time; stops after a short idle period.
- Recipient's client acknowledges a message → system updates that message's status for the sender.

Functional Requirements

ID	Requirement	Priority	Verification
FR-007	The system shall allow an authenticated user to send a direct text message to another registered user.	High	Functional test
FR-008	The system shall deliver a direct message to the recipient's client in real time over WebSocket when the recipient is online.	High	Integration test
FR-009	The system shall persist a direct message addressed to an offline recipient and deliver it automatically the next time that recipient connects.	High	Integration test
FR-010	The system shall emit a typing-indicator event to the other participant while the sender is actively composing a message, and clear it after a short idle timeout.	Medium	Functional test

FR-011	The system shall track and expose each direct message's status as one of: sent, delivered, or read.	High	Unit + integration test

5.3 Group / Room Chat

Description and Priority

Lets users organize into named rooms and exchange group messages broadcast to every connected member. Priority: High. Owned by Satvik L.

Stimulus/Response Sequences

- User requests to create a room with a unique name → system creates the room and makes the creator its first admin.
- User requests to join/leave a room → system updates room membership and notifies existing members.
- A room member sends a message → system broadcasts it to every other currently-connected member of that room.

Functional Requirements

ID	Requirement	Priority	Verification
FR-012	The system shall allow an authenticated user to create a chat room with a unique room name, becoming that room's first admin.	High	Functional test
FR-013	The system shall allow an authenticated user to join an existing room or leave a room they currently belong to.	High	Functional test
FR-014	The system shall broadcast a group message in real time to every member of that room who is currently connected.	High	Integration test
FR-015	The system shall allow only current members of a room to view or send messages within that room.	High	Negative/security test
FR-016	The system shall require every room to retain at least one admin at all times, rejecting an action that would leave it with none.	Medium	Negative test

5.4 Message History & Status Tracking

Description and Priority

Persists every message and exposes it back to clients as searchable, paginated history so conversations survive reconnects and logouts. Priority: High/Medium. Jointly supported by Prithvi Aithal (direct) and Satvik L (group).

Functional Requirements

ID	Requirement	Priority	Verification
FR-017	The system shall persist every direct and group message with its sender, timestamp, and conversation/room reference.	High	Integration test
FR-018	The system shall allow a user to retrieve the message history of a conversation or room in reverse-chronological, paginated order.	High	Functional test

5.5 Room Administration

Description and Priority

Gives a room's admin(s) control over who belongs to the room and who else holds admin rights. Priority: Medium. Owned by Satvik L.

Functional Requirements

ID	Requirement	Priority	Verification
FR-019	The system shall allow a room admin to promote another current member of that room to admin.	Medium	Functional test
FR-020	The system shall allow a room admin to remove a member from that room.	Medium	Functional test
FR-021	The system shall reject a room-administration action attempted by a member who does not hold the admin role for that room.	High	Authorization test

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	95% of chat messages shall be delivered to an online recipient within 300 ms under the defined test load.	Latency/load test
NFR-002	Performance	95% of REST API responses (login, history) shall complete within 1 second under the defined test load.	Performance test

6.2 Safety Requirements

ChatSphere is a text-communication tool with no physical actuation, so conventional safety-of-life requirements do not apply. The only safety-adjacent concern is data safety: the system shall persist a message to the database before acknowledging it as ‘sent’ to the sender, so that a server crash immediately after send cannot silently lose an already-acknowledged message.

6.3 Security Objectives and Requirements

Security Objectives

These objectives state the security goals that the requirements below are designed to satisfy; each requirement traces to at least one objective.

- SEC-OBJ-1 (Confidentiality of credentials): protect user credentials and session tokens from disclosure, both at rest (password hashes, never plaintext) and in transit (HTTPS/WSS only), so that an attacker who observes traffic or accesses the database cannot recover a usable password or forge a session.
- SEC-OBJ-2 (Authorization integrity): ensure every direct message, room message and room-administration action is only ever performed by, or visible to, a user who is authenticated and holds the correct role (room member or room admin) for that action, preventing privilege escalation and cross-room data leakage.
- SEC-OBJ-3 (Input and content safety): ensure no user-supplied content (messages, usernames, room names) can be used to inject executable script (XSS) or otherwise corrupt data integrity for other users of the system.
- SEC-OBJ-4 (Availability under abuse): ensure a single connected client cannot degrade service for other users through message flooding or connection abuse, and that security-relevant events are observable after the fact via logging.

Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall never be stored in plaintext; only bcrypt password hashes shall be persisted.	Code/configuration review
SEC-002	Role-based access control shall prevent a non-admin room member from performing room-administration actions.	Authorization tests
SEC-003	All user-supplied message content shall be sanitized/escaped before storage and before rendering, to prevent stored XSS.	Negative/security tests

SEC-004	REST endpoints and WebSocket connections shall reject expired or invalid JWTs.	Security test
SEC-005	The system shall rate-limit message-send events per connection to mitigate spam/flood abuse.	Load/security test
SEC-006	Security-relevant events (failed login attempts, unauthorized room-admin attempts) shall be logged.	System test

6.4 Software Quality Attributes

- Maintainability – business logic is separated into independent, unit-tested services (auth, messaging, room, presence).
- Reliability – a client that reconnects within 60 seconds of a dropped socket resumes without losing any queued message.
- Usability – a new user can register and send their first message within 3 primary screens.
- Testability – core socket event handlers expose a testable interface independent of the live Socket.IO transport, so they can be unit tested with mocked sockets.
- Portability – the server is packaged as a Docker image and configured entirely via environment variables.

6.5 Business Rules

- Only current members of a room may send or read messages in that room.
- Only a room's admin may promote/demote members or remove a member from that room.
- A direct-message conversation exists only between two distinct registered users.
- A password must satisfy a minimum complexity rule (at least 8 characters) before registration succeeds.
- A room must retain at least one admin at all times.

Also include Domain requirements here: message ordering within a single conversation/room must be strictly timestamp-consistent so history always renders in the order messages were actually sent.

7. Other Requirements

- Database: message history is retained indefinitely in this phase; a future release may add user-initiated deletion/retention limits.
- Legal/localization: no legal or localization requirements apply to this academic-project phase; UI text is English-only.
- Reuse: the project reuses open-source libraries (Express, Socket.IO, Mongoose, JWT, bcrypt, Redis client) under their respective permissive (MIT-style) licenses; no proprietary third-party components are used.

8. Validation Specification – Initial Planning

At the Requirements stage, this validation specification is planning-focused: it explains how each category of requirement above will eventually be validated, including planned security/penetration-style testing. It is not the final test report — concrete pass/fail results and the completed Requirement Traceability Matrix (Appendix C) will be produced once Deliverables 3 and 4 (Design, Implementation & Testing) are complete.

Requirement Category	Planned Validation Approach	Example	Timing / Owner
Functional	Black-box functional tests derived from each FR in Section 5	Verify FR-008: message delivered in real time when recipient is online	Sprints 2–4, feature owner
Security	Authentication/authorization tests and negative/security tests derived from Section 6.3	Verify SEC-002: non-admin member is rejected from room-admin actions	Sprint 5, Sourish Ganesh

Performance	Load/latency testing against the NFR-001/NFR-002 targets	Confirm 95% message delivery within 300 ms under defined concurrent load	Sprint 5, all members
Usability	Task-based walkthrough by a first-time user	New user registers and sends a first message within 3 primary screens	Sprint 4, all members
Reliability	Failure/retry and reconnect scenario testing	Client reconnecting within 60 s resumes without losing a queued message	Sprints 4–5, all members
Unit	Automated unit tests with coverage measurement per module (auth, messaging, room, presence)	Message status transition logic (sent → delivered → read)	Sprints 2–5, all members

Planned penetration-style checks: manual attempts to call room-admin REST/WebSocket actions with a non-admin JWT, an expired JWT and a tampered JWT, plus a check that message content is escaped before storage/render to rule out stored XSS. Results will be recorded in the System Validation Report at Deliverable 5.

9. Project Plan

The project follows an Agile (Scrum-style), sprint-based lifecycle across six two-week sprints, chosen because the feature set (auth, direct messaging, room chat, presence, admin) decomposes naturally into independently demoable increments that map to Section 10 of the Synopsis. Work is tracked as user stories/tasks on GitHub Projects (Jira-equivalent), with Git/GitHub commit history checked as evidence of individual contribution.

9.1 Work Breakdown Structure (WBS)

Work Package	Tasks	Owner	Effort (person-days)	Output
WP1	Requirements gathering, use cases, SRS baseline	All	4	SRS
WP2	High-level architecture + UML (use case, class, sequence, component)	Satvik L / Sourish Ganesh	4	Architecture / Design doc
WP3	Authentication & Presence module	Sourish Ganesh	6	Working feature
WP4	Direct Messaging module	Prithvi Aithal	6	Working feature
WP5	Group / Room Chat module	Satvik L	7	Working feature
WP6	Message history, status tracking, room administration	Prithvi Aithal / Satvik L	6	Working feature
WP7	Testing + CI/CD + security validation	All	6	Test / validation evidence
WP8	Final system validation + consolidated report	All	4	Final report

Total estimated effort: 43 person-days (~14 person-days per member) across a 12-week, six-sprint schedule.

9.2 Schedule (Sprint / Gantt Summary)

The table below is the text-form schedule; the team maintains the visual Gantt chart / burndown as a GitHub Projects board (screenshot attached as evidence in Deliverable 4).

Sprint	Weeks	Work Packages Active

Sprint 1	Weeks 1–2	WP1 (complete); WP2 (start)
Sprint 2	Weeks 3–4	WP2 (complete); WP3 (start)
Sprint 3	Weeks 5–6	WP3 (complete); WP4
Sprint 4	Weeks 7–8	WP5 (start); WP6 (start)
Sprint 5	Weeks 9–10	WP5, WP6 (complete); WP7
Sprint 6	Weeks 11–12	WP7 (complete); WP8

9.3 Tool Stack

Activity	Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL, commit history
Agile planning	GitHub Projects (Jira-equivalent)	Backlog, user stories, sprint board
CI/CD	GitHub Actions	Workflow run screenshot/log
Containerization	Docker	Dockerfile + build/run evidence
Testing	Jest / Supertest (backend)	Test results
API testing	Postman	Collection / results
Static analysis	SonarQube or similar	Analysis report
Diagramming	draw.io / PlantUML	UML diagrams

Appendix A: Glossary

Term	Definition
Socket / WebSocket	A persistent, full-duplex TCP-based connection that lets the server push events to a client without the client polling.
Room	A named group conversation that one or more registered users can join to exchange group messages.
Presence	A user's real-time online/offline connection state, broadcast to their contacts and rooms.
JWT	JSON Web Token – a signed token used to authenticate a user on REST calls and the WebSocket handshake.
Typing Indicator	A transient real-time event shown to a conversation's other participant(s) while a message is being composed.
Read Receipt	A status update shown to a sender indicating the recipient has viewed their message.
RBAC	Role-Based Access Control – restricting an action based on a user's assigned role (e.g. room admin vs member).
ODM	Object-Document Mapper – here, Mongoose, which maps MongoDB documents to JavaScript objects/schemas.

Appendix B: Field Layouts

Derived from the entity-relationship model in Figure 2, the core persisted fields are:

User

Field	Length	Data Type	Description	Mandatory
user_id	24	ObjectId/String	Unique user identifier (primary key)	Y

username	30	String	Unique display/login handle	Y
email	100	String	Unique email address	Y
password_hash	60	String	bcrypt hash of the user's password	Y
status	10	Enum	online / offline	Y

Room

Field	Length	Data Type	Description	Mandatory
room_id	24	ObjectId/String	Unique room identifier (primary key)	Y
name	50	String	Unique room name	Y
type	10	Enum	direct / group	Y
created_by	24	ObjectId/String	FK → User who created the room	Y

RoomMembership

Field	Length	Data Type	Description	Mandatory
membership_id	24	ObjectId/String	Unique membership identifier (primary key)	Y
room_id	24	ObjectId/String	FK → Room	Y
user_id	24	ObjectId/String	FK → User	Y
role	10	Enum	member / admin	Y

Message

Field	Length	Data Type	Description	Mandatory
message_id	24	ObjectId/String	Unique message identifier (primary key)	Y
room_id	24	ObjectId/String	FK → Room (direct conversations use a 2-member room)	Y
sender_id	24	ObjectId/String	FK → User who sent the message	Y
content	2000	String	Message text body	Y
timestamp	–	DateTime	Server-assigned send time	Y
status	10	Enum	sent / delivered / read	Y

Appendix C: Requirement Traceability Matrix

At the Requirements stage this matrix carries requirement IDs only; the Architecture, Design, Code and Test-Case reference columns will be completed once Deliverable 3 (Design) and Deliverable 4 (Implementation & Testing) are finalized, per the course's stage-wise validation-specification flow.

Sl. No	Requirement ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test Case ID
1	FR-001	Unique user registration	TBD	TBD	TBD	TBD	TBD
2	FR-007	Send direct message	TBD	TBD	TBD	TBD	TBD
3	FR-012	Create room	TBD	TBD	TBD	TBD	TBD
4	FR-014	Broadcast group message	TBD	TBD	TBD	TBD	TBD
5	SEC-002	Room-admin RBAC	TBD	TBD	TBD	TBD	TBD

Appendix D: As-Built Changes (Nov 2026)

This section records how the delivered system differs from the previous version of this document. Each change is explained in docs/DEVIATIONS.md in the project repository (IDs in brackets).

Requirements: No requirement was removed or weakened. FR-001 to FR-021, NFR-001, NFR-002 and SEC-001 to SEC-006 are delivered as specified; the changes above are additions or clarifications.

Process: Six two-week sprints became two one-week sprints (19–23 Oct, 26–30 Oct) plus a Sprint 3 (9–15 Nov) for spill-over and Part-3 testing (D-01, D-08). Team work was redistributed during Sprint 3.

Data model: Message gains an idempotencyKey (unique per sender) so retried sends are not duplicated (D-03). Message content is stored HTML-escaped; the 2000-character limit applies to the stored form (D-11).

API: Additive Socket.IO events and acks: message:read ack, typing:update with username, session:ready, room:joined / room:left / room:restore, RATE_LIMITED on the message:send ack (D-05, D-06, D-07, D-09). Additive REST endpoints: user and room search, room members (D-14). Failed logins return 401 AUTH_INVALID_TOKEN with one message for every failure (D-10). Full reference: docs/API.md.

Implementation choices: bcryptjs (same bcrypt algorithm and hash format) instead of the native bcrypt binding (D-12). Presence is broadcast to a user's direct-message contacts, tracked per socket in Redis, with a 1 s / 3 s Socket.IO heartbeat so a drop is detected within 5 s (D-14). In Sprint 1 a DM was started by user ID; user and room search replaced that in Sprint 3 (D-04).

Verification: All 29 requirements are implemented and traced to code and tests (docs/RTM.md). 244/244 automated tests pass; all 23 Test Plan v1.1 cases pass (docs/reports/system-validation-report.md); NFR-001 p95 delivery 16.2 ms (target 300 ms), NFR-002 p95 REST 66.1 ms (target 1 s) (docs/reports/load-test.md). Test cases were updated in Test Plan v1.1 (D-13).